



# PROGRAMAÇÃO



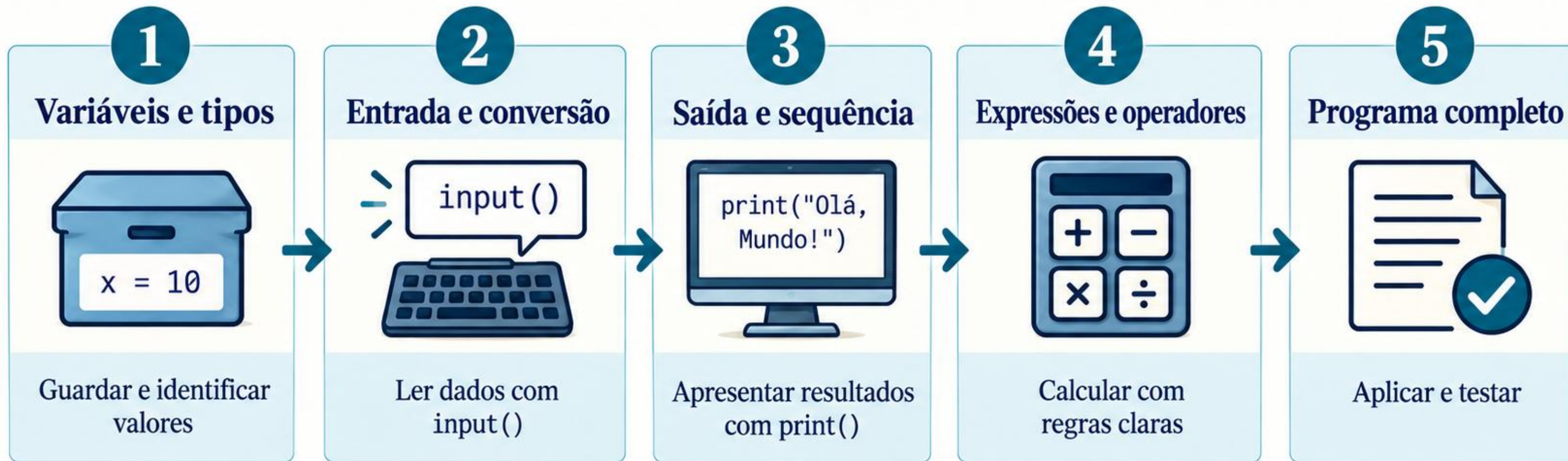
Escola Naval



António Gonçalves

# AGENDA DA AULA

## Programação • Escola Naval



Conceito



exemplo



aplicação



verificação

# ENTRADA → PROCESSAMENTO → SAÍDA



## Exemplo com valores definidos no código

```
distancia = 120  
velocidade = 15
```



```
tempo = distancia / velocidade
```



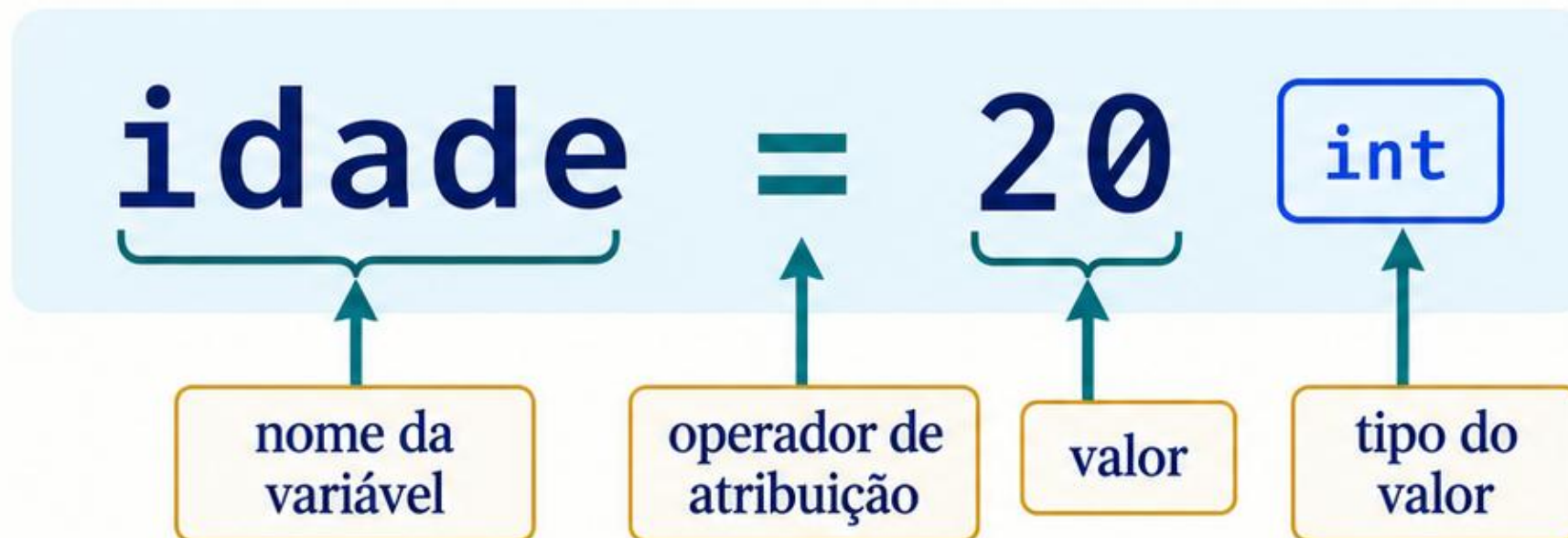
```
print(tempo)  
8.0
```

Um programa transforma dados de entrada em resultados.

# Variáveis e atribuição

**Definição** | Uma variável associa um nome a um valor.

## Anatomia de uma variável



## Exemplo em Python

```
nome = "Ana"      altura = 1.68      aprovado = True
```

# ATUALIZAÇÃO E RASTREIO

## Exemplo de código (Python)

```
numero = 10
numero = 20
numero = numero + 5
```

## Tabela de rastreo

| ETAPA   | INSTRUÇÃO EXECUTADA   | VALOR DE "numero" |
|---------|-----------------------|-------------------|
| Etapa 1 | numero = 10 ✓         | 10                |
| Etapa 2 | numero = 20 →         | 20                |
| Etapa 3 | numero = numero + 5 → | 25                |



**IDEIA-CHAVE:** O valor atual resulta das instruções já executadas.

# NOMES DE VARIÁVEIS EM PYTHON

Escolhe nomes válidos, claros e significativos.



## NOMES VÁLIDOS

Seguem as regras e são usados corretamente.



**idade**

Nome simples e claro.



**nome\_aluno**

Usa sublinhado para separar palavras.



**nota\_final**

Descritivo e fácil de perceber.



## NOMES INVÁLIDOS

Não seguem as regras do Python.

**123**

**2idade**

Começa por número.



**nome aluno**

Contém espaço.



**nota-final**

Contém hífen.



**class**

Palavra reservada da linguagem.



## REGRA PRINCIPAL

Letras, algarismos e sublinhado.  
O primeiro carácter não pode ser um algarismo.



## MAIÚSCULAS E MINÚSCULAS DISTINGUEM NOMES

São variáveis diferentes.

**idade = 16**

idade

16

**Idade = 20**

Idade

20



Preferir nomes claros:

**velocidade\_media**

em vez de

**v.**

# TIPOS DE DADOS

PYTHON NA ESCOLA NAVAL

## int — números inteiros



Números sem parte decimal.

20      0      -5

**Exemplo:** 20 é um número inteiro (int).

## str — texto



Sequências de caracteres (letras, números, símbolos).

"Ana"      "20"

**Exemplo:** "20" é texto (str), não é um número.

## float — números com parte decimal



Números reais, com parte decimal.

1.75      12.5      -0.5

**Exemplo:** 1.75 é um número com parte decimal (float).

## bool — valores lógicos



Representa valores de verdade.



True      False

**Exemplo:** True e False são valores lógicos (bool).



O tipo do valor determina as operações possíveis.

# type() – VERIFICAR O TIPO

## Exemplo de código

```
idade = 16
altura = 1.72
nome = "Ana"
aprovado = True

print(type(idade))
print(type(altura))
print(type(nome))
print(type(aprovado))
```

## A INSPECÇÃO COM type()



## RESULTADOS DA INSPECÇÃO

| Variável | Tipo de Dados (Output) |
|----------|------------------------|
| idade    | → <class 'int'>        |
| altura   | → <class 'float'>      |
| nome     | → <class 'str'>        |
| aprovado | → <class 'bool'>       |



## ATENÇÃO

**type()** apenas informa;  
não altera o valor.



## IDEIA-CHAVE

**type()** permite confirmar como Python está a interpretar um valor.

# O QUE É `input()`?

```
nome = input("Nome: ")
```

## `input()`



apresenta uma mensagem;

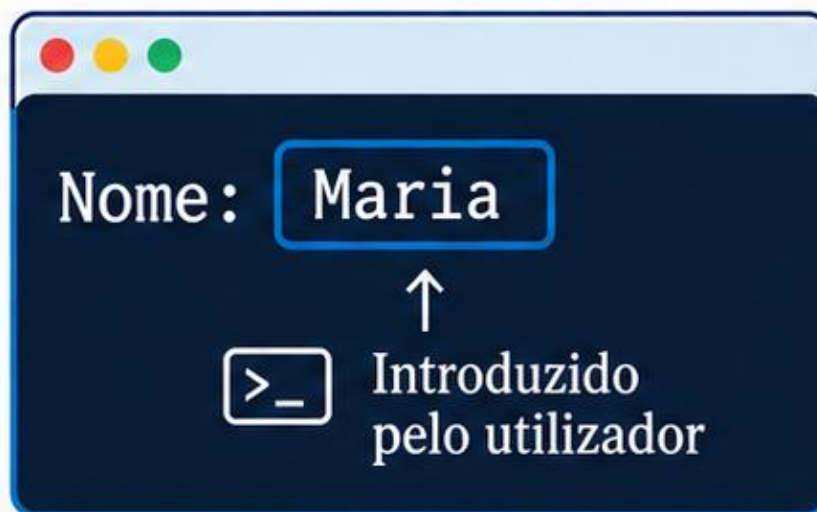


espera pela resposta;



devolve aquilo que o utilizador escreveu.

## Terminal



## Resultado



nome

→ "Maria"

## IDEIA-CHAVE

**`input()`** é a função principal para ler dados introduzidos pelo utilizador.

# input() DEVOLVE SEMPRE TEXTO

## Explicação

A função **input()** lê uma entrada do utilizador através do teclado e **devolve sempre um texto**.

| Entrada do Utilizador                                                                                           | Execução                                                                                        | RESULTADO                                                 |
|-----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
|  <p>Entrada do Utilizador</p> | <p>↑ EXECUÇÃO ↓</p> <pre>idade = input("Idade: ")</pre> <p>Qual é o tipo de <b>idade</b>? ?</p> | <pre>idade = "16"</pre> <p>↑<br/>tipo:<br/><b>str</b></p> |



## Conclusão

Mesmo quando o utilizador escreve números, **input()** devolve **texto**.

# CONVERSÃO DE TIPOS

A conversão cria um valor do tipo pretendido quando o conteúdo é compatível.

## 1 DE TEXTO PARA INTEIRO

Converte uma cadeia de caracteres num número inteiro.

texto

"16"



inteiro

16

```
>>> int("16")  
# resultado  
16
```

## 2 DE TEXTO PARA DECIMAL

Converte uma cadeia de caracteres num número decimal (ponto flutuante).

texto

"12.5"



decimal

12.5

```
>>> float("12.5")  
# resultado  
12.5
```

## 3 DE INTEIRO PARA TEXTO

Converte um número inteiro numa cadeia de caracteres.

inteiro

16



texto

"16"

```
>>> str(16)  
# resultado  
"16"
```



ESCOLHA  
PRÁTICA



Contagens: `int()`



Medidas: `float()`



Mensagens: `str()`

# ERROS DE CONVERSÃO

## Conversões válidas

```
int("16")
```



16



```
float("16.5")
```



16.5



## Conversões inválidas

```
int("dezasseis")
```



ValueError



```
int("16.5")
```



ValueError



```
float("16,5")
```



ValueError



O texto tem de ser compatível com o tipo pretendido.



Nos números decimais, Python usa ponto: 16.5

# LER E CONVERTER NUMA INSTRUÇÃO

Qual função é executada primeiro?

| 1. input("Idade: ")                                                                                                                                                  | 2. utilizador escreve "16"                                                                                            | 3. input() devolve "16"                                                                                                                                 | 4. int("16")                                                                                                                    | 5. resultado = 16                                                             | 6. idade recebe 16                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <pre>idade = int(input("Idade: "))</pre> <p>A função <b>input()</b> é executada primeiro.<br/>Mostra <b>"Idade: "</b> no ecrã e espera a resposta do utilizador.</p> |  <p>O utilizador escreve "16".</p> |  <p>A função <b>input()</b> devolve a cadeia de caracteres "16".</p> | <p>16 → <code>int("16")</code> → 16</p> <p>A função <b>int()</b> converte a cadeia de caracteres "16" no número inteiro 16.</p> | <p>→ <b>16</b> →</p> <p>O resultado da conversão é 16 (o número inteiro).</p> | <pre>idade = 16</pre> <p>A variável <b>idade</b> recebe o valor 16.</p> |

**IDEIA-CHAVE**



Numa expressão combinada, a leitura ocorre antes da conversão.

# SAÍDA COM `print()`

## Código Python

```
print("Olá")  
print(42)  
nome = "Ana"  
print("Nome:", nome)
```



## Saída no ecrã



Apresenta texto e valores



A vírgula separa  
argumentos



Por defeito, separa argumentos  
com espaços e termina a linha

# TEXTO, VARIÁVEIS E EXPRESSÕES

```
nome = "Ana"
```

|                                                                                                         | Instrução                      | Saída               |
|---------------------------------------------------------------------------------------------------------|--------------------------------|---------------------|
|  Texto literal         | <code>print("nome")</code> ➡   | <code>nome</code>   |
|  Valor da variável    | <code>print(nome)</code> ➡     | <code>Ana</code>    |
|  Expressão calculada | <code>print(10 + 5)</code> ➡   | <code>15</code>     |
|  Texto literal       | <code>print("10 + 5")</code> ➡ | <code>10 + 5</code> |



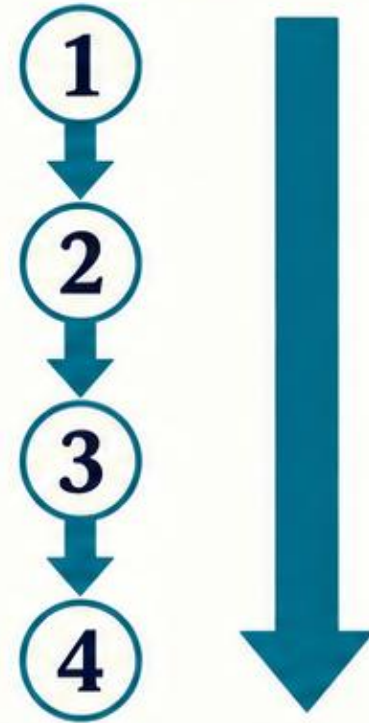
As aspas distinguem texto literal de código a avaliar.

# O QUE É EXECUÇÃO SEQUENCIAL?

## Código Python (instruções)

```
a = 10  
b = 5  
resultado = a + b  
print(resultado)
```

## Ordem de execução



## Saída do programa



Nesta fase, as instruções são executadas uma após outra.



**IDEIA-CHAVE:** Num programa sequencial, Python executa as instruções de cima para baixo.

# Rastrear várias variáveis

## Código Python

```
1 x = 5
2 y = 10
3 z = x + y
4 x = x + 10
```

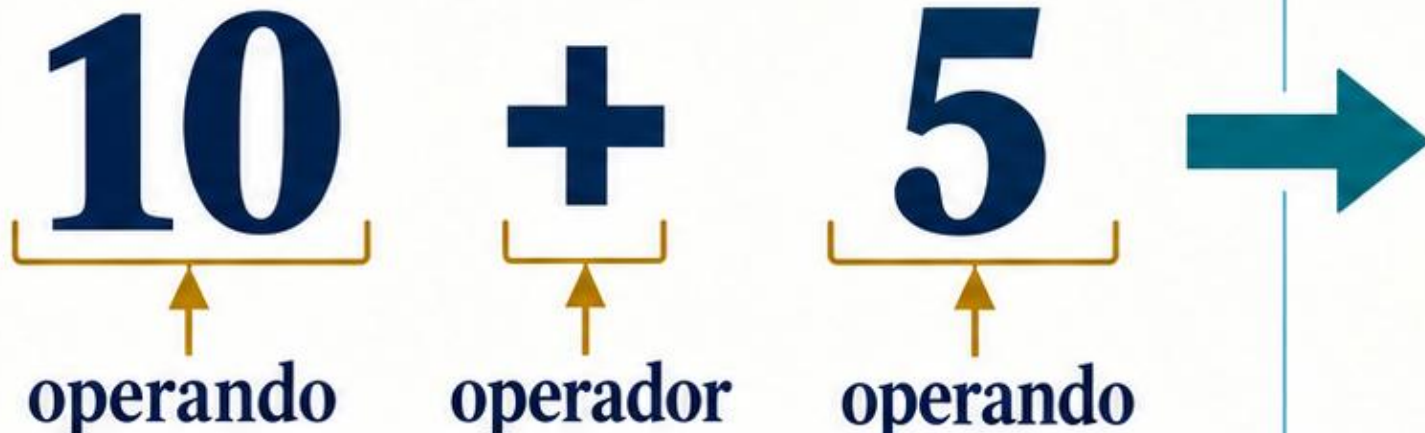
## Estado das variáveis após cada instrução

|   |                                     | x             | y             | z             |
|---|-------------------------------------|---------------|---------------|---------------|
| ➔ | Após instrução<br><b>x = 5</b>      | <b>x = 5</b>  | <b>y = —</b>  | <b>z = —</b>  |
| ➔ | Após instrução<br><b>y = 10</b>     | <b>x = 5</b>  | <b>y = 10</b> | <b>z = —</b>  |
| ➔ | Após instrução<br><b>z = x + y</b>  | <b>x = 5</b>  | <b>y = 10</b> | <b>z = 15</b> |
| ➔ | Após instrução<br><b>x = x + 10</b> | <b>x = 15</b> | <b>y = 10</b> | <b>z = 15</b> |



Porque z continua 15?

# EXPRESSÕES, OPERADORES E OPERANDOS

| Expressão                  | Elementos da expressão                                                                                                | Resultado              |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>10 + 5</b><br>expressão | <br>operando   operador   operando | <b>15</b><br>resultado |



Uma expressão combina valores e operadores para produzir um resultado.

**IDEIA-CHAVE:**

Expressão = operandos + operadores ➡ resultado.



# OPERADORES ESPECIAIS: //, %, \*\*



## Divisão inteira

Devolve o **quociente** da divisão inteira, ignorando o resto.

**Exemplo:**

$$17 \text{ // } 5 \rightarrow 3$$

quociente

$$17 = 3 \times 5 + 2$$



## Resto

Devolve o resto da divisão inteira.

**Exemplo:**

$$17 \% 5 \rightarrow 2$$

resto

$$17 = 3 \times 5 + 2$$



## Potência

Eleva o primeiro operando à potência do segundo.

**Exemplo:**

$$2 ** 3 \rightarrow 8$$

$$2 \times 2 \times 2 = 8$$



### Nota importante:

Arredonda o quociente para baixo.

$$-7 \text{ // } 2 = -4$$

# Tipos e Operadores

ESCOLA NAVAL

| Conceito                                                                                                   | Exemplo em Python           | Resultado                   | Explicação                                              |
|------------------------------------------------------------------------------------------------------------|-----------------------------|-----------------------------|---------------------------------------------------------|
|  Soma numérica             | <code>10 + 5</code> →       | <code>15</code>             | Soma dois números e devolve um resultado numérico.      |
|  Concatenação de texto     | <code>"10" + "5"</code> →   | <code>"105"</code>          | Junta duas cadeias de texto e devolve um único texto.   |
|  Repetição de texto       | <code>"Python" * 2</code> → | <code>"PythonPython"</code> | Repete o texto o número indicado de vezes.              |
|  Tipos incompatíveis     | <code>"10" + 5</code> →     | <code>TypeError</code>      | Não é possível somar uma cadeia de texto com um número. |
|  Operação numérica mista | <code>10 + 2.5</code> →     | <code>12.5</code>           | Soma um número inteiro com um número decimal.           |



O comportamento de um operador depende dos tipos dos operandos.

# PRECEDÊNCIA E PARÊNTESES

1

## Parênteses ( )

As expressões entre parênteses são calculadas primeiro.



$$2 + 3 * 4 = 14$$

$$3 * 4 = 12$$

2

## Potência \*\*

A potência tem precedência sobre multiplicação, divisão, adição e subtração.



$$2 ** 3 = 8$$

3

## Multiplicação e divisão: \* / // %

A multiplicação, a divisão, a divisão inteira (//) e o resto da divisão (%) têm a mesma precedência.



$$(2 + 3) * 4 = 20$$

$$2 + 3 = 5$$

4

## Adição e subtração: + -

A adição e a subtração têm a mesma precedência.



Ao mesmo nível, calcular da esquerda para a direita. A potência associa da direita para a esquerda.

$$2 ** 3 ** 2 = 2 ** (3 ** 2) = 512$$

# Do problema ao programa

## Exercício final: Duração de uma viagem

Um navio viaja a velocidade constante. Calcular a duração em horas.

### ENTRADA



**distancia:** milhas náuticas  
**velocidade:** nós

### PROCESSAMENTO



`tempo = distancia / velocidade`

### SAÍDA



Tempo de viagem em horas.

Escrever um programa que leia os dois valores com `float(input(...))`, calcule tempo e apresente o resultado com `print()`.



Considerar distância não negativa e velocidade maior que zero.

**Teste:** 120 milhas náuticas e 15 nós

**Resultado esperado:** 8.0 horas